



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

**Faculty of
Computing**

SECR2043 OPERATING SYSTEMS

SESSION 2024/2025 SEMESTER 2

SECTION 3

GROUP PROJECT

NON-PREEMPTIVE SCHEDULING

SJF, PRIORITY SCHEDULING

LECTURER'S NAME: DR. DANLAMI GABI

GROUP NAME: ERROR 404

NO.	GROUP MEMBERS	MATRIC NO
1	TAN ZHI MING	A23CS0189
2	LEE YIN SHEN	A23CS0236
3	SOU CHENG JIE	A23CS0271
4	TAI YI TIAN	A23CS0272

Table of Contents

Print Queue System using Shortest Job First (SJF).....	3
1.0 Abstract.....	3
2.0 Introduction.....	3
3.0 Problem Statement.....	3
4.0 Related Works.....	4
5.0 Methodology.....	4
5.1 Flowchart.....	4
5.2 Hardware and Software Requirements.....	5
5.3 Task Distribution.....	5
6.0 Design & Implementation.....	6
6.1 System Flowchart.....	6
6.2 Code Implementation.....	6
7.0 Results & Discussion.....	10
7.1 Interface and Execution.....	10
7.2 Output Analysis.....	11
7.3 Discussion.....	12
8.0 Future Work & Conclusion.....	13
8.1 Future Work.....	13
8.2 Conclusion.....	13
Airport Runway Scheduling using Priority Scheduling.....	14
1.0 Abstract.....	14
2.0 Introduction.....	14
3.0 Problem Statement.....	14
4.0 Related Works.....	15
5.0 Methodology.....	15
5.1 Flowchart.....	15
6.0 Design & Implementation.....	17
6.1 System Flowchart.....	17
6.2 Code Implementation.....	17
7.0 Results & Discussion.....	20
7.1 Interface and Execution.....	20
7.2 Output Analysis.....	21
7.3 Discussion.....	22
8.0 Future Work & Conclusion.....	23
8.1 Future Work.....	23
8.2 Conclusion.....	23
9.0 Acknowledgement.....	24
10.0 References.....	24

Print Queue System using Shortest Job First (SJF)

1.0 Abstract

The Shortest Job First (SJF) scheduling algorithm is a non-preemptive scheduling method that manages tasks by giving priority to the shortest tasks. In this project, we apply the SJF algorithm to a print queue system, where the goal is to sort the printing jobs based on the number of pages for each job. This scheduling method aims to reduce the average waiting time by giving priority to the smaller printing jobs. By simulating this system, the project shows how SJF improves the overall efficiency of the print queue system, and also points out that the longer jobs may be delayed if more shorter jobs keep coming.

2.0 Introduction

The management of printing jobs in an office or other places is important to ensure the efficient use of resources and minimize the waiting time for each job. The Shortest Job First (SJF) algorithm is widely recognized for its ability to prioritize shorter tasks that can improve overall system efficiency. So, in this project, we apply SJF to a print queue system to simulate how print jobs are managed. The simulation is to explore the practical application of this algorithm in real-world scenarios, which is a printer that receives multiple jobs, and to evaluate its performance in reducing job turnaround time and waiting time.

3.0 Problem Statement

Managing printing jobs efficiently is a challenge in the situation where multiple tasks need to be processed sequentially. The main issue that arises when printing jobs with different sizes is submitted, it will lead to inefficient task handling and longer waiting times for larger jobs. The objective of this project is to implement and evaluate the Shortest Job First (SJF) scheduling algorithm in a print queue system. By simulating this system, we will understand how SJF can optimize the scheduling of print jobs, reduce average waiting times, and highlight the potential drawbacks like job starvation for longer tasks.

4.0 Related Works

In this project, we will be simulating 2 non-preemptive scheduling algorithms which are Shortest Job First(SJF) and Priority Scheduling. These algorithms are commonly introduced in operating systems due to their straightforward logic and use in real-life scenarios where task interruption is not allowed.

SJF selects the job with the shortest time first. It helps reduce the average waiting time but may cause starvation in longer jobs if shorter jobs keep arriving.

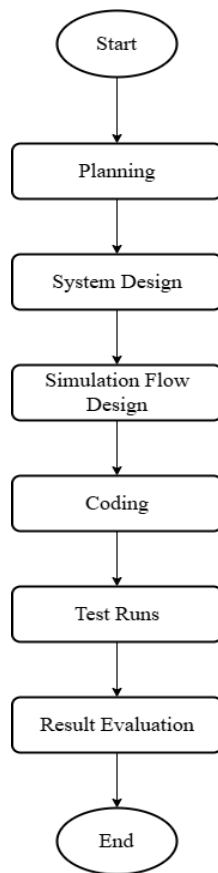
Priority Scheduling selects jobs based on priority levels, where a lower number indicates higher priority. It is suitable for systems like aircraft landing and hospital emergency rooms but may lead to starvation for low priority tasks.

These two scheduling methods are simple yet effective in specific environments, and this project will help us understand how they behave when applied in practical scenarios.

5.0 Methodology

5.1 Flowchart

To complete the simulation of non-preemptive scheduling algorithms, our team followed a series of structured phases. These steps included planning, system design, simulation flow design, coding, test runs and result evaluation. Here is the entire workflow illustrated in flowchart:



Explanation of Each Phase:

Planning: We began by selecting 2 non-preemptive scheduling algorithms which are SJF and Priority Scheduling. For the real-world scenarios, we decided to choose the print queue for SJF and the airport runway system for Priority Scheduling.

System Design: We found out how the system would interact with the user. This included input format, required job and how the output will be displayed.

Coding: Both simulations were implemented in C++. The logic followed the scheduling rules without using any external libraries to keep the implementation simple.

Test Runs: We ran several test cases with different sets of inputs to verify that the simulations produced the correct execution order.

Result Evaluation: The final outputs were reviewed to ensure the scheduling logic was being applied correctly and results followed the non-preemptive scheduling concept.

5.2 Hardware and Software Requirements

Software: Visual Studio Code as the IDE for C++ coding, [Draw.io](https://draw.io) as the flowchart drawing tool.

Hardware: A standard laptop capable of running VS code and compiling C++ code.

5.3 Task Distribution

Research: Lee Yin Shen & Sou Cheng Jie

Simulations Coding: Tan Zhi Ming & Tai Yi Tian

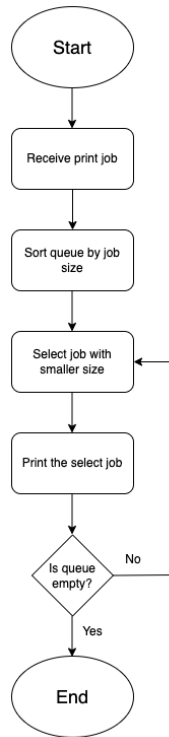
Testing & Debugging: Tan Zhi Ming & Tai Yi Tian

Report: All Members

6.0 Design & Implementation

Print Queue System

6.1 System Flowchart



6.2 Code Implementation

Initialization

The code is implemented with c++. The PrintJob struct is defined to store attributes like id, pages needed to be printed and the job arrival time.

```
#include <iostream>
#include <vector>

using namespace std;

struct PrintJob {
    string id;
    int pages;
    int arrivalTime;
};
```

Receive print jobs

The system will ask the number of print jobs that need to be processed. After that, the user can enter the job id for each job and how many pages it needs to be printed. The arrival time of each job will be auto assigned based on the order of users enter the job.

```
int n;
cout << "Enter the number of print jobs: ";
cin >> n;

vector<PrintJob> jobs(n);

for (int i = 0; i < n; ++i) {
    cout << "\nEnter job ID" << i + 1 << ": ";
    cin >> jobs[i].id ;
    cout << "Enter number of pages for job " << i + 1 << ": ";
    cin >> jobs[i].pages;
    jobs[i].arrivalTime = i;
}
```

Sort Algorithm

Each print job will be compared, jobs with fewer pages are scheduled first. If the pages of two print jobs are the same, first come first serve (FCFS) will be used to sort the job.

```
bool compareSJF(const PrintJob &a, const PrintJob &b) {
    if (a.pages == b.pages)
        return a.arrivalTime < b.arrivalTime;
    return a.pages < b.pages;
}
```

```
// Sort jobs based on SJF criteria
sort(jobs.begin(), jobs.end(), compareSJF);
```

Output Displaying

For the result display, it will display the job from fewer pages to more pages which contain information such as job id, pages and the time that each job takes.

```
cout << "\n--- Print Order (Non-Preemptive SJF) ---\n";
int currentTime = 0;
for (const PrintJob &job : jobs) {
    cout << "Printing " << job.id << " (" << job.pages << " pages) from time "
        << currentTime << " to " << (currentTime + job.pages) << ".\n";
    currentTime += job.pages;
}
```

Complete Simulation Code

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  struct PrintJob {
7      string id;
8      int pages;
9      int arrivalTime;
10 };
11
12 bool compareSJF(const PrintJob &a, const PrintJob &b) {
13     if (a.pages == b.pages)
14         return a.arrivalTime < b.arrivalTime;
15     return a.pages < b.pages;
16 }
17
18 int main() {
19     int n;
20     cout << "Enter the number of print jobs: ";
21     cin >> n;
22
23     vector<PrintJob> jobs(n);
24
25     for (int i = 0; i < n; ++i) {
26         cout << "\nEnter job ID" << i + 1 << ": ";
27         cin >> jobs[i].id;
28         cout << "Enter number of pages for job " << i + 1 << ": ";
29         cin >> jobs[i].pages;
30         jobs[i].arrivalTime = i;
31     }
32
33     // Sort jobs based on SJF criteria
34     sort(jobs.begin(), jobs.end(), compareSJF);
35
36     cout << "\n--- Print Order (Non-Preemptive SJF) ---\n";
37     int currentTime = 0;
38     for (const PrintJob &job : jobs) {
39         cout << "Printing " << job.id << " (" << job.pages << " pages) from time "
40             << currentTime << " to " << (currentTime + job.pages) << ".\n";
41         currentTime += job.pages;
42     }
43
44     return 0;
45 }
```

7.0 Results & Discussion

7.1 Interface and Execution

The interface of the print queue system is designed as a simple text-based console application that allows users to simulate the scheduling of print jobs using the Shortest Job First (SJF) algorithm. The execution flow involves collecting input from the user regarding the number of print jobs, their IDs, and the number of pages for each job. Once all job information is entered, the system sorts the jobs according to the SJF criteria and displays the print schedule.

The interface provides a clear and interactive sequence of prompts, allowing users to enter data step-by-step and observe the job execution timeline. This simulation helps visualize how SJF prioritizes smaller print jobs to reduce waiting time.

The key steps involved in the execution include:

- *User Input Collection*: Prompting the user to enter the total number of print jobs, followed by the job ID and number of pages for each individual job.
- *Arrival Time Assignment*: Automatically assigning arrival times based on the input sequence to simulate the order of job submission.
- *SJF Sorting*: Reordering the jobs using the SJF logic—jobs with fewer pages are placed earlier in the queue. Ties are broken using the job arrival time.
- *Print Job Execution*: Simulating the print process by displaying the start and end time for each job in the scheduled order.
- *Output Display*: Printing a summary of the print order and execution timeline in the console, allowing users to understand the impact of SJF scheduling.

7.2 Output Analysis

The output generated by the system clearly reflects the behavior of the SJF algorithm. Regardless of the input order, jobs with fewer pages are moved to the front of the queue. The program displays the print schedule with time stamps indicating when each job starts and finishes. For instance, if four jobs are entered in the order A (5 pages), B (2 pages), C (8 pages), and D (1 page), the output is:

```
Enter the number of print jobs: 4
Enter job ID1: A
Enter number of pages for job 1: 5
Enter job ID2: B
Enter number of pages for job 2: 2
Enter job ID3: C
Enter number of pages for job 3: 8
Enter job ID4: D
Enter number of pages for job 4: 1
--- Print Order (Non-Preemptive SJF) ---
Printing D (1 pages) from time 0 to 1.
Printing B (2 pages) from time 1 to 3.
Printing A (5 pages) from time 3 to 8.
Printing C (8 pages) from time 8 to 16.
```

This output confirms that the system correctly reorders the jobs based on page count and calculates the cumulative execution time accurately for each job.

7.3 Discussion

The simulation results demonstrate that the SJF scheduling algorithm can effectively optimize print order by prioritizing smaller tasks. This leads to more efficient processing and improved responsiveness for short jobs. The key insights from the output analysis include:

- *Reduced Average Waiting Time:*
Shorter jobs complete earlier, which helps reduce the average waiting time across all print tasks and improves overall system responsiveness.
- *Job Reordering Based on Size:*
The algorithm overrides the input order, ensuring jobs are handled based on page count. This shows its potential for optimizing job throughput in systems with size variability.
- *Starvation of Larger Jobs:*
Jobs with more pages may face longer delays, particularly if a continuous stream of smaller jobs enters the system. This risk of starvation is a known limitation of SJF.
- *Static Scheduling Assumption:*
The system assumes that all jobs are submitted at once. For real-world use where jobs may arrive dynamically, enhancements like preemptive scheduling or job queues would be needed.
- *Ease of Use:*
The simple input structure and clear output make the system intuitive and easy to test. It serves well as a simulation tool to understand the behavior of non-preemptive scheduling algorithms.

8.0 Future Work & Conclusion

8.1 Future Work

To further enhance the current system and improve its realism and usability, the following future developments are recommended:

- **Dynamic Job Arrival Handling:** Simulate real-time job submissions with varying arrival times, creating a more realistic and interactive scheduling scenario.
- **Hybrid Scheduling with Priorities:** Introduce user-defined priorities to complement job size, allowing critical print tasks to override SJF rules when necessary.
- **Parallel Printer Simulation:** Extend the system to simulate multiple printers working concurrently, introducing multithreaded scheduling logic.
- **Performance Metrics Summary:** Automatically calculate and display key metrics such as average waiting time, turnaround time, and total print duration for analytical insights.

8.2 Conclusion

This project successfully demonstrates the effectiveness of the Shortest Job First (SJF) algorithm in optimizing print job scheduling by minimizing total waiting time and prioritizing shorter tasks. The simulation confirms that SJF improves responsiveness for small jobs and offers a practical understanding of non-preemptive scheduling logic. However, the system's static nature and potential for job starvation highlight areas for future refinement. Overall, this simple yet functional simulation serves as a valuable educational tool for illustrating scheduling principles in operating systems.

Airport Runway Scheduling using Priority Scheduling

1.0 Abstract

Priority Scheduling is a non-preemptive scheduling algorithm that is used to manage tasks based on their priority levels. In this project, we apply Priority Scheduling to simulate the allocation of runway time at an airport. The aircraft with higher priorities, like emergency flight, will be given priority over the lower priority flights like commercial and private planes. By simulating this system, the project will show how Priority Scheduling helps to manage the runway time more effectively but also points out that less urgent flights may be delayed if more important flights keep arriving.

2.0 Introduction

Managing flight landings and takeoffs in an airport is very important to make sure all operations run smoothly and avoid the delay of flights. The Priority Scheduling allows the flights to be prioritized based on their priority level. This project will focus on simulating the runway scheduling using Priority Scheduling, where the priority of each flight determines the order of runway usage. By simulating this system, we can know the impact of this scheduling algorithm on flight management, especially in scenarios that involve emergencies, and to see how it can handle conflicts when multiple flights share the same priority level.

3.0 Problem Statement

Managing the runway time in an airport can be complex especially when dealing with a mix of emergency, commercial, and private flights. The current problem in airport management is to efficiently allocate the runway time to ensure that the flights, especially emergency ones, are given the highest priority, while still ensuring that commercial and private flights are well handled. This project is to implement and evaluate a Priority Scheduling algorithm to simulate the allocation of runway time. The goal is to explore how well Priority Scheduling can handle

different flight types, minimize delays for high-priority flights, and to see how this scheduling method can be applied in real-world air traffic control scenarios.

4.0 Related Works

In this project, we will be simulating 2 non-preemptive scheduling algorithms which are Shortest Job First(SJF) and Priority Scheduling. These algorithms are commonly introduced in operating systems due to their straightforward logic and use in real-life scenarios where task interruption is not allowed.

SJF selects the job with the shortest time first. It helps reduce the average waiting time but may cause starvation in longer jobs if shorter jobs keep arriving.

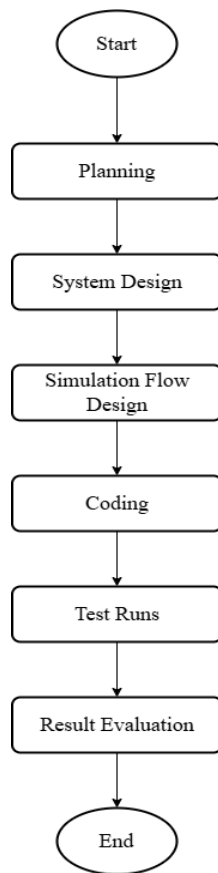
Priority Scheduling selects jobs based on priority levels, where a lower number indicates higher priority. It is suitable for systems like aircraft landing and hospital emergency rooms but may lead to starvation for low priority tasks.

These two scheduling methods are simple yet effective in specific environments, and this project will help us understand how they behave when applied in practical scenarios.

5.0 Methodology

5.1 Flowchart

To complete the simulation of non-preemptive scheduling algorithms, our team followed a series of structured phases. These steps included planning, system design, simulation flow design, coding, test runs and result evaluation. Here is the entire workflow illustrated in flowchart:



Explanation of Each Phase:

Planning: We began by selecting 2 non-preemptive scheduling algorithms which are SJF and Priority Scheduling. For the real-world scenarios, we decided to choose the print queue for SJF and the airport runway system for Priority Scheduling.

System Design: We found out how the system would interact with the user. This included input format, required job and how the output will be displayed.

Coding: Both simulations were implemented in C++. The logic followed the scheduling rules without using any external libraries to keep the implementation simple.

Test Runs: We ran several test cases with different sets of inputs to verify that the simulations produced the correct execution order.

Result Evaluation: The final outputs were reviewed to ensure the scheduling logic was being applied correctly and results followed the non-preemptive scheduling concept.

5.2 Hardware and Software Requirements

Software: Visual Studio Code as the IDE for C++ coding, [Draw.io](https://draw.io) as the flowchart drawing tool.

Hardware: A standard laptop capable of running VS code and compiling C++ code.

5.3 Task Distribution

Research: Lee Yin Shen & Sou Cheng Jie

Simulations Coding: Tan Zhi Ming & Tai Yi Tian

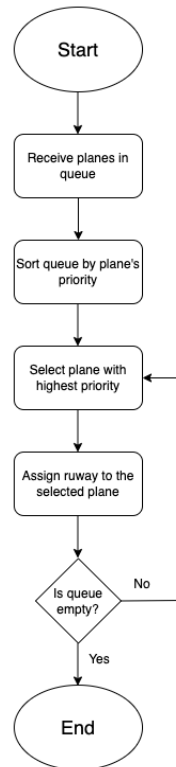
Testing & Debugging: Tan Zhi Ming & Tai Yi Tian

Report: All Members

6.0 Design & Implementation

Airport Runway Scheduling

6.1 System Flowchart



6.2 Code Implementation

Initialization

The Plane struct is defined to store flight id, priority, runway time and the original arrival order.

```
#include <iostream>
#include <vector>

using namespace std;

struct Plane {
    string flightID;
    int priority;           // Lower value = higher priority
    int runwayTime;        // Time it takes on runway
    int arrivalOrder;      // Original order of arrival
};
```

Receive plane queue

First, users need to enter the number of planes. After that, the system will ask the users to enter information for each plane such as the flight id, priority and the runway time needed. The original arrival order will be auto assigned based on the order of the user key in.

```
int n;
cout << "Enter number of planes in queue: ";
cin >> n;

vector<Plane> queue;

// Input planes
for (int i = 0; i < n; i++) {
    Plane p;
    cout << "\nEnter details for Plane " << i + 1 << ":\n";
    cout << "Flight ID: ";
    cin >> p.flightID;
    cout << "Priority (1=Emergency, 2=Commercial, 3=Private): ";
    cin >> p.priority;
    cout << "Runway time needed (in minutes): ";
    cin >> p.runwayTime;
    p.arrivalOrder = i;
    queue.push_back(p);
}
```

Sort algorithm using priority scheduling

The function will sort the planes based on their priority. Planes with higher priority will be scheduled first. If there are planes with the same priority, first come first serve rules will be apply.

```
// Comparator for sorting by priority (then arrival order)
bool compareByPriority(const Plane &a, const Plane &b) {
    if (a.priority == b.priority)
        return a.arrivalOrder < b.arrivalOrder;
    return a.priority < b.priority;
}
```

```
// Sort by priority (non-preemptive)
sort(queue.begin(), queue.end(), compareByPriority);
```

Output Displaying

For the result display, it will display the plane from high priority to low priority which contains information such as plane id, priority and the time that each plane takes.

```
cout << "\n--- Runway Scheduling Order (Non-Preemptive Priority) ---\n";
int currentTime = 0;
for (const Plane &p : queue) {
    cout << "Plane " << p.flightID << " (Priority " << p.priority << ") is using the runway from minute "
        << currentTime << " to " << (currentTime + p.runwayTime) << ".\n";
    currentTime += p.runwayTime;
}
```

Complete Simulation Code

```
1  #include <iostream>
2  #include <vector>
3
4  using namespace std;
5
6  struct Plane {
7      string flightID;
8      int priority;      // Lower value = higher priority
9      int runwayTime;   // Time it takes on runway
10     int arrivalOrder;  // Original order of arrival
11 };
12
13 // Comparator for sorting by priority (then arrival order)
14 bool compareByPriority(const Plane &a, const Plane &b) {
15     if (a.priority == b.priority)
16         return a.arrivalOrder < b.arrivalOrder;
17     return a.priority < b.priority;
18 }
19
20 int main() {
21     int n;
22     cout << "Enter number of planes in queue: ";
23     cin >> n;
24
25     vector<Plane> queue;
26
27     // Input planes
28     for (int i = 0; i < n; i++) {
29         Plane p;
30         cout << "\nEnter details for Plane " << i + 1 << ":\n";
31         cout << "Flight ID: ";
32         cin >> p.flightID;
33         cout << "Priority (1=Emergency, 2=Commercial, 3=Private): ";
34         cin >> p.priority;
35         cout << "Runway time needed (in minutes): ";
36         cin >> p.runwayTime;
37         p.arrivalOrder = i;
38         queue.push_back(p);
39     }
40
41     // Sort by priority (non-preemptive)
42     sort(queue.begin(), queue.end(), compareByPriority);
43
44     cout << "\n--- Runway Scheduling Order (Non-Preemptive Priority) ---\n";
45     int currentTime = 0;
46     for (const Plane &p : queue) {
47         cout << "Plane " << p.flightID << " (Priority " << p.priority << ") is using the runway from minute "
48             << currentTime << " to " << (currentTime + p.runwayTime) << ".\n";
49         currentTime += p.runwayTime;
50     }
51
52     return 0;
53 }
```

7.0 Results & Discussion

7.1 Interface and Execution

The interface of the runway scheduling system is implemented as a simple console application, which uses a non-preemptive priority scheduling algorithm to simulate aircraft scheduling. It guides users through step-by-step input, allowing for easy simulation and observation of how aircraft are managed in response to emergencies.

The system accepts flight data, including flight ID, priority and the required runway time. When two or more flights share the same priority, each aircraft will be automatically assigned an arrival order to resolve conflicts. The scheduling mechanism ensures that high-priority aircraft (for example, aircraft in emergency situations) enter first and will not be interrupted once they start using the runway.

The key steps involved in the execution include:

- *User Input Collection:* The user is prompted to enter the number of planes, followed by each plane's flight ID, priority (1=Emergency, 2=Commercial, 3=Private), and runway usage time.
- *Arrival Order Assignment:* Each plane is assigned a unique arrival sequence number during input.
- *Priority Sorting:* The queue is sorted based on the priority level, with ties resolved by arrival order to maintain fairness.
- *Runway Scheduling Simulation:* The program displays the exact timing each plane occupies the runway, based on the cumulative execution time.

7.2 Output Analysis

The simulation produces a sequential log showing the order in which each plane uses the runway, based on the priority-first logic. For example, an emergency flight (priority 1) will always precede commercial or private ones, regardless of when it arrived. If two flights have the same priority, the one that arrived earlier is scheduled first.

```
Enter number of planes in queue: 4

Enter details for Plane 1:
Flight ID: A
Priority (1=Emergency, 2=Commercial, 3=Private): 3
Runway time needed (in minutes): 10

Enter details for Plane 2:
Flight ID: B
Priority (1=Emergency, 2=Commercial, 3=Private): 2
Runway time needed (in minutes): 15

Enter details for Plane 3:
Flight ID: C
Priority (1=Emergency, 2=Commercial, 3=Private): 1
Runway time needed (in minutes): 20

Enter details for Plane 4:
Flight ID: D
Priority (1=Emergency, 2=Commercial, 3=Private): 3
Runway time needed (in minutes): 4

--- Runway Scheduling Order (Non-Preemptive Priority) ---
Plane C (Priority 1) is using the runway from minute 0 to 20.
Plane B (Priority 2) is using the runway from minute 20 to 35.
Plane A (Priority 3) is using the runway from minute 35 to 45.
Plane D (Priority 3) is using the runway from minute 45 to 49.
```

This output behavior effectively mirrors real-world airport protocols where urgency of the situation (emergencies) takes precedence over other factors. The simulation also ensures non-preemption, meaning that once a plane begins using the runway, it retains exclusive access until it completes its required time.

7.3 Discussion

The implementation successfully demonstrates the effectiveness and fairness of Non-Preemptive Priority Scheduling for managing shared critical resources like airport runways. Key observations include:

- **Emergency flights are always prioritized**, minimizing risks and ensuring safety.
- **Commercial and private flights are queued based on arrival** only when priority is tied, which ensures orderly and fair execution.
- **Non-preemptive nature ensures smooth runway allocation**, eliminating mid-execution interruptions that could complicate real-world operations.
- **Simple structure and stable performance** make this scheduling suitable for systems where task durations are predictable and preemption is either undesirable or unsafe.

Although this model is highly suitable for environments with fixed task times and urgent task processing, it may not be applicable under high-load conditions because low-priority tasks may experience starvation. Therefore, future adaptations may include aging mechanisms or preemptive expansions to enhance the fairness of dynamic real-time systems.

8.0 Future Work & Conclusion

8.1 Future Work

While the current simulation effectively models runway scheduling based on static, non-preemptive priority, several enhancements can be introduced to handle more dynamic and realistic airport conditions:

- **Aging Mechanism:** Implement aging to gradually boost the priority of waiting for lower-priority flights, preventing starvation during high-traffic periods.
- **Multiple Runways Simulation:** Upgrade the system to handle multiple runways concurrently, reflecting operations in large international airports.
- **Real-Time Flight Arrivals:** Extend the system to simulate dynamic arrival of flights over time, allowing the scheduler to adjust the queue as new data comes in.

8.2 Conclusion

This system clearly demonstrates the practical application of non-preemptive priority scheduling in the management of key resources at airports. By implementing strict priority rules and simulating dedicated runways for each flight, it captures the essence of the key to safety in real-world air traffic control. The clear structure, the fairness of the connection processing carried out through the arrival sequence, and the consistency with the emergency priority protocol make it suitable for controlled airport scenarios. However, in order to adapt to dynamic or crowded environments, enhanced functions such as aging or multi-runway management will be indispensable. Overall, this model provides an educational and practical basis for exploring real-time scheduling systems in the aviation environment.

9.0 Acknowledgement

We would like to express our sincere appreciation to our SECR2043 Operating System lecturers, Dr. Muhammad Zafran Bin Muhammad Zaly Shah and Dr. Danlami Gabi for providing clear and helpful lectures throughout the semester. The lectures they provided not only helped us in understanding the theory inside the slides and textbook, but also guided us in applying those concepts to real-life examples through this project.

The explanations on SJF and Priority Scheduling made it easier for us to build our simulations step by step with confidence. We are truly grateful for their effort, support and all the learning materials shared during this course.

We would also like to thank all our group members for their teamwork, commitment and cooperation in completing this project successfully.

10.0 References

- GeeksforGeeks. (n.d.). *Preemptive and Non-Preemptive Scheduling*. GeeksforGeeks.
<https://www.geeksforgeeks.org/operating-systems/preemptive-and-non-preemptive-scheduling/>
- TutorialsPoint. (n.d.). *Shortest Job First (SJF) Scheduling*. TutorialsPoint.
https://www.tutorialspoint.com/operating_system/os_shortest_job_first_scheduling.htm
- Sanfoundry. (n.d.). *C Program for SJF Scheduling*. Sanfoundry.
<https://www.sanfoundry.com/c-program-sjf-scheduling/>
- GeeksforGeeks. (n.d.). *Priority Scheduling in Operating System*. GeeksforGeeks.
<https://www.geeksforgeeks.org/priority-scheduling-in-operating-system/>
- TutorialsPoint. (n.d.). *C++ Program for Priority Scheduling*. TutorialsPoint.
<https://www.tutorialspoint.com/cplusplus-program-for-priority-scheduling>